Homework 2 - Multiple Threads (5 points)

Due: April 11, 11:30 PM

Submission Requirements

You need to submit a compressed archive of your homework directory. Under your homework directory, you should include:

- All your source code.
- A Makefile for compiling the program.
- A README file that describes how to compile and test your program.

Before submitting, create a gzipped tarball (.tar.gz) of your entire homework directory using the following command:

tar -czvf username-homework-2.tar.gz your_homework_directory/

where username is your NETID.

Submit the tarball on Canvas.

Problem

Level 1

Implement a program that calculates the sum of squares of numbers from 1 to 100 using 2 to 5 threads.

For example, if using two threads:

- The first thread can compute: $1^2+2^2+3^2+\cdots+50^2$
- The second thread can compute: $51^2+52^2+53^2+\cdots+100^2$

Each thread should add its partial sum to a shared global variable (sum), which is initialized to 0.

To simulate workload, each thread should wait for a random time between 0 to 1 second before performing its calculations. (Hint: To sleep a fraction of a second, call usleep().)

⚠ Warning: The rand() function is not thread-safe. Students can either:

- Use random() with srandom() to generate thread-safe random numbers.
- Use rand() but protect it with a mutex to avoid race conditions..

Requirements:

- Since multiple threads access the shared variable sum, you must prevent race conditions using appropriate synchronization techniques (e.g., mutex or semaphore).

- The main thread should wait for all worker threads to complete before printing the final sum.
- Ensure your code is readable and robust.
- Print standard Linux error messages if an error occurs (e.g., use perror("Thread creation failed") if a thread cannot be created).

Level 2

Enhance your program to allow user input for:

- The end number n
- The number of threads

For example, the command:

./main 200 10

should calculate the sum $1^2 + 2^2 + ... + 200^2$ using 10 threads for subset additions.

Input Constraints:

- The end number n must be between 10 and 1000 (inclusive).
- The number of threads must be between 2 and n/2 (inclusive), with a maximum of 100 to avoid overloading the system.
- If the input values are out of range, the program should exit with an error message.

Synchronization Considerations

- This assignment focuses on multiple threads. Do not use the direct mathematical formula n(n+1)(2n+1)/6 to compute the sum.
- Since multiple threads modify a shared global variable (sum), you must implement synchronization to prevent race conditions.
- Acceptable synchronization methods include:
  - Mutex (pthread_mutex_t) – Protect access to the shared variable.
  - Semaphore (sem_t) – Control access to the shared variable across threads.
- The main thread must synchronize with all worker threads before printing the final result.

Level 3 (Optional - Producer-Consumer Model)

Extend your implementation to use a producer-consumer model where:

- A producer generates numbers and adds them to a shared queue.
- Multiple consumer threads process these numbers and compute their squares.

(You may implement level 3 in a separate main program to reduce code complexity.)

Additional Requirements for Level 3:

- The number of consumer threads should be configurable.
- Use a bounded queue (preferably a circular buffer) to hold numbers awaiting processing.
- Producers and consumers should be synchronized properly to avoid race conditions.
- You may choose your synchronization approach using mutexes and/or semaphores (or condition variable), as long as the implementation is correct.
- The producer should also sleep for a random time to simulate variable workload.

Implementation Tips:

- The producer thread adds numbers to the queue, and consumer threads retrieve and process them.
- Use a mutex to protect shared access to the queue.
- Two semaphores can help coordinate access:
  - One semaphore to track available items in the queue (increases when the producer adds items, decreases when a consumer retrieves them).
  - Another semaphore to track remaining space in the queue (decreases when the producer adds, increases when a consumer retrieves).
- Consumers should process numbers from the queue until all tasks are completed.

Objective:

Even if you do not implement this level, reading and understanding the producer-consumer model specification will still contribute to your learning.

General Notes:

- Proper synchronization must be used to avoid race conditions and ensure correctness.
- Deadlocks and busy waiting should be avoided.
- The queue should be managed efficiently, and you should decide on the best way to handle full/empty states.
- Use standard Linux error handling mechanisms where necessary.

This assignment provides a practical understanding of thread synchronization, concurrent programming techniques, and managing shared data between threads.